Laboratório Funcional · Aula 12

Johan Gabriel da Silva dos Santos · RA 24190316-2 · Turma B
Gerado em 18/10/2026, 20:57:24 · tempo registrado no cronômetro: 60:31

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

para cada uma, o que a impede de ser pura.

1. agora(): É impura porque depende do relógio do sistema, podendo retornar resultados diferentes em cada chamada.
2. registra(msg): É impura porque utiliza print() para escrever na tela, causando um efeito colateral.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: 1: A-1 A-2
Linha 4: 4: 14
Linha 7: 7: [7, 3, 9] [3, 7, 9]
A função proximo_id incrementa o contador a cada chamada, gerando A-1 e A-2. Na composição, g(4) = 3 × 4 = 12 e f(12) = 12 + 2 = 14. Já sorted(notas) cria uma nova lista ordenada sem alterar a lista original.

Saídas (original e com compor(g, f))

```
1° execução:
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]

Com compor(g, f):
1: A-1 A-2
2: True
3: 8
4: 18
5: [4, 9, 16]
6: 29
```

```
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes porque utiliza uma variável global (contador), que é incrementada a cada chamada. Isso dificulta os testes porque o resultado depende do estado anterior do programa, tornando os testes menos previsíveis e reproduzíveis.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Temos f(x) = x + 2 e g(x) = 3x. Na composição compor(f, g)(4), primeiro calculamos g(4) = 12 e depois f(12) = 14. Já em compor(g, f)(4), primeiro calculamos f(4) = 6 e depois g(6) = 18. Os resultados são diferentes porque a ordem de aplicação das funções foi invertida.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

A função sorted(notas) retorna uma nova lista ordenada sem modificar a lista original, enquanto notas.sort() ordena a própria lista, modificando-a diretamente. sorted(notas) segue melhor o estilo funcional porque evita alterar os dados originais, tornando o código mais previsível.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

B C

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
1° Execução:
utput:
22
14
25
8
negativo
```

```
A
(B C)
(A B C)
((A B) C D)
B



2° Execução:
22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 9b388618c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 9b3…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A

Debug this error
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') serve para indicar que uma expressão deve ser tratada como dado, sem ser executada. Sem ele, o Scheme tentou interpretar (A B C) como uma chamada de função, considerando A como a função e B e C como parâmetros. Como A não estava definida, ocorreu o erro Unbound variable: A.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
expressão (car (cdr (cdr '(A B C)))) devolve o terceiro elemento da lista. Resultado no OneCompiler: C.
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

A função conta-ate em Scheme pode conseguir contar até um milhão se o interpretador otimizar a recursão em cauda, reutilizando os registros de ativação. Já em Python, a função provavelmente não conseguirá chegar a um milhão de chamadas, pois o Python possui um limite de profundidade de recursão e poderá gerar um erro RecursionError antes de terminar.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
# Prática 3b - A mesma recursão em cauda, agora em Python
# Cole em https://onecompiler.com/python
def conta_ate(n, acc):
    if n == 0:
        return acc
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação

print(conta_ate(500, 0))
print(conta_ate(1_000_000, 0))
Exit 1
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
```

```
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded

Debug this error
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a pilha de execução, pois cada chamada precisa guardar operações pendentes para realizar quando as chamadas seguintes terminarem. Na recursão em cauda, não ficam operações pendentes após a chamada recursiva, permitindo que a linguagem reutilize o registro de ativação e mantenha o uso de memória constante quando existe otimização.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme conseguiu executar um milhão de chamadas porque otimiza a recursão em cauda, reutilizando os registros de ativação e evitando o crescimento da pilha. Já o Python não realiza essa otimização automaticamente e possui um limite de profundidade de recursão, por isso a execução termina com RecursionError antes de alcançar um milhão.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
1° execução:
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))


2° execução:
tamanho: 3  ok
```

```
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
A função tamanho verifica se a lista está vazia usando null?; se estiver, retorna 0. Caso contrário, soma 1 e
chama a função novamente com cdr, que representa o restante da lista. A função soma-quadrados também verifica se
a lista está vazia; se não estiver, utiliza car para obter o primeiro elemento, calcula seu quadrado e soma ao
resultado da chamada recursiva com cdr. A função pares verifica se a lista está vazia, utiliza car para obter o
primeiro elemento e even? para verificar se ele é par. Se for par, utiliza cons para adicioná-lo ao resultado da
chamada recursiva; caso contrário, ignora o elemento e continua percorrendo o restante com cdr
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como percorrer a lista recursivamente e combinar os resultados de cada chamada. Usei null? para identificar o fim da lista, car para acessar o primeiro elemento e cdr para continuar a operação com os elementos restantes. Na função pares, também precisei utilizar even? para selecionar apenas os números pares e cons para construir a lista resultante.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16], pois calcula apenas os cinco primeiros elementos da lista infinita de quadrados. Já primeiro 42 (div 1 0) devolve 42, porque Haskell utiliza avaliação preguiçosa e a função primeiro ignora o segundo parâmetro, portanto a divisão por zero nunca é calculada.

Saídas (original e com member 17)

```
1° execução:
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

2° execução:
Main.hs:26:3: error:
```

```
    • Couldn't match expected type: (a1 → IO ()) → Bool → IO ()
                  with actual type: IO ()
    • The function ‘print’ is applied to three value arguments,
        but its type ‘a0 → IO ()’ has only one
      In a stmt of a 'do' block:
        print (primeiro 42 (div 1 0)) print (member 17 squares)
      In the expression:
        do print (fact 20)
           print (factors 28)
           print (sort [7, 2, ....])
           print (take 5 squares)
           ....
   |
26 |   print (primeiro 42 (div 1 0))
   |   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^...
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita porque utiliza avaliação preguiçosa, calculando apenas os elementos necessários para produzir o resultado solicitado. A expressão primeiro 42 (div 1 0) não gera erro porque a função primeiro ignora o segundo parâmetro, então a divisão por zero nunca é avaliada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

A busca por member 17 squares nunca termina porque 17 não é um quadrado perfeito e a função continua percorrendo a lista infinita. Já member 16 squares termina porque encontra o número 16 na lista. Para corrigir isso, podemos interromper a busca quando o elemento atual for maior que o valor procurado, já que a lista está em ordem crescente. Assim, a função pode ser escrita como member b [] = False; member b (a:x) | a > b = False | a == b = True | otherwise = member b x.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) vai imprimir 15, pois soma 10 com 5. Já [1, 2, 3].map(vezes(5)) vai imprimir [5, 10, 15], pois multiplica cada elemento da lista por 5.

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida por somador(10) mantém acesso à variável n, que vale 10, por meio de um fechamento (closure). Mesmo depois que somador termina de executar, o ambiente da função continua disponível enquanto a função devolvida existir, permitindo que mais10(5) some 10 e 5.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque compose aplica primeiro g e depois f, enquanto andThen aplica primeiro f e depois g. Assim, f.compose(g).apply(4) calcula f(g(4)) = f(12) = 14, enquanto f.andThen(g).apply(4) calcula g(f(4)) = g(6) = 18. O método compose corresponde a compor(f, g) da Prática 1, pois executa primeiro g e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
```

```
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
o exercício 1 imprime 244, pois soma os quadrados dos números pares (8, 12 e 6). O exercício 2 imprime ANA,
CARLA, DAVI, pois seleciona os alunos com nota maior ou igual a 6 e transforma seus nomes em letras maiúsculas.
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional pode ser mais fácil de ler porque separa claramente as operações de filtrar, transformar e somar os dados. Já a versão com laço pode ser mais fácil de depurar quando precisamos acompanhar cada etapa ou verificar valores intermediários, pois permite observar como as variáveis mudam durante a execução.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque expressa diretamente as operações de gerar os números, elevar cada um ao cubo e somar os resultados, enquanto a versão em C exige acompanhar o laço e a variável acumuladora. Eu aceitaria o custo de execução maior em sistemas de análise de dados ou relatórios, nos quais a clareza e a facilidade de manutenção são importantes, mas evitaria esse custo em sistemas de tempo real ou aplicações que exigem alto desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi descobrir que a avaliação preguiçosa do Haskell permite trabalhar com listas infinitas sem calcular todos os seus elementos.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está na expressão b <- h, que deveria ser b <= h, pois a intenção é filtrar os elementos menores ou iguais ao pivô. A correção é sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]. O compilador interpreta b <- h como uma tentativa de percorrer h como uma lista, mas h é o pivô, um elemento individual, e não uma lista. Por isso, ocorre um conflito de tipos entre o elemento e uma lista.